

Universidad ORT Uruguay

Facultad de Ingeniería

Bernard Wand Polak



Documentación - Obligatorio 1

ESTRUCTURA DE DATOS Y ALGORITMOS 2

N4A - 2025

Marcelo Rezzano - 195348
Dominique Lachaise - 214233

Docente: Joaquin Vigna

ÍNDICE

Tabla de resultados.....	3
Descripción de las soluciones:.....	4
Ejercicio 1 - Emails.....	4
Ejercicio 2 - Diccionario.....	4
Ejercicio 3 - Heapify.....	5
Ejercicio 4 - Camino más corto.....	5
Ejercicio 5 - Ciclicidad en grafos.....	6
Declaraciones generales y pruebas:.....	7
Ejercicio 1 - Emails:.....	8
Ejercicio 2 - Diccionario:.....	8
Ejercicio 3 - Heapify:.....	8
Ejercicio 4 - Camino más corto:.....	9
Ejercicio 5 - Ciclicidad de grafos:.....	9

Tabla de resultados

Problema	Resultado
1	Completo
2	Completo
3	Completo
4	Completo
5	Completo
6	No implementado
7	No implementado
8	No implementado
9	No implementado
10	No implementado

Descripción de las soluciones:

Ejercicio 1 - Emails

Nombre del archivo: ejercicio1.cpp

Explicación de la solución: Se utiliza una tabla hash abierta para insertar todos los correos electrónicos, utilizando la dirección como clave y valor. Para implementar esta tabla se utilizó la estructura ListImp.cpp que fue provista en el template del obligatorio.

Luego para la implementación de de la solución, se agrega un contador de elementos repetidos que se incrementa al momento de la inserción de la tabla, luego tenemos el método unicos() que le resta el contador a la cantidad total de elementos, devolviendo así la cantidad de claves distintas.

Complejidad temporal: La complejidad esperada es $O(N)$ en promedio, ya que las operaciones de inserción en la tabla hash abierta son $O(1)$ en promedio.

Estructuras utilizadas:

- Tabla Hash Abierta
- ListImp

Declaración de autoría: Declaramos que esta solución fue implementada por nuestro grupo de forma original. Utilizando los tads y el template brindados en clase.

La función de hash utilizada en esta solución fue tomada del material proporcionado en clase.

Ejercicio 2 - Diccionario

Nombre del archivo: ejercicio2.cpp

Explicación de la solución: Se implementa una tabla hash cerrada con doble hashing. Se insertan las palabras del diccionario y luego con el método existe() se busca cada palabra dentro de la tabla de hash retornando 0 o 1 respectivamente.

Complejidad temporal: En promedio, tanto la inserción como la búsqueda tienen complejidad $O(1)$ debido al uso de doble hashing.

Estructuras utilizadas:

- Tabla Hash Cerrada

Declaración de autoría: Declaramos que esta solución fue implementada por nuestro grupo de forma original. Utilizando los tads y el template brindados en la clase.

En esta solución, una de las funciones de hash fue tomada del material brindado en clase, y la segunda fue tomada de un obligatorio anterior cursado por Dominique Lachaise, integrante del equipo.

Ejercicio 3 - Heapify

Nombre del archivo: ejercicio3.cpp

Explicación de la solución: Se insertan los elementos en una estructura de heap mínimo sin ordenarlos, utilizando una función especial de inserción `InsertarSinOrdenar()` que simplemente inserta directamente en el array que representa el heap.

Luego se aplica la función `heapify()` para convertir el array en un min-heap. Finalmente, se eliminan los elementos uno a uno desde el tope para imprimirlos en orden.

Complejidad temporal: La construcción del heap con `heapify` tiene complejidad $O(N)$. La extracción de los elementos uno por uno tiene complejidad $O(\log N)$ por operación. En total, $O(N \log N)$.

Estructuras utilizadas:

- MinHeap con `heapify`

Declaración de autoría: Declaramos que esta solución fue implementada por nuestro grupo de forma original.

Ejercicio 4 - Camino más corto

Nombre del archivo: ejercicio4.cpp

Explicación de la solución: Se modela el grafo como una lista de adyacencias. Para resolver el problema, se implementa el algoritmo de Dijkstra usando una cola de prioridad eficiente en nuestro caso un MinHeap.

Para esto se modificó la implementación del Heap ya utilizado en el ejercicio 3, agregando el método de insertar y pasando una función de comparación por el constructor.

En el Heap se guarda un nodo `Cam` que representa el vértice y su costo, usando el costo como ponderación dentro del MinHeap.

Por cada vértice solicitado en la entrada, se calcula la distancia mínima a todos los nodos y se imprimen los resultados. Si el vértice destino no es alcanzable o es el mismo vértice de origen, se imprime -1.

Complejidad temporal:

La complejidad esperada es $O(N \times (V + E) \log V)$, donde N es la cantidad de vértices origen, V la cantidad total de vértices, y E la cantidad de aristas.

Estructuras utilizadas:

- Lista de adyacencias para modelar el grafo
- MinHeap para Dijkstra

Declaración de autoría:

Declaramos que esta solución fue implementada por nuestro grupo de forma original, utilizando los TADs brindados en clase.

Ejercicio 5 - Ciclicidad en grafos

Nombre del archivo: ejercicio5.cpp

Explicación de la solución: Se modela el grafo como una lista de adyacencias. Para detectar si existen ciclos, aplicamos ordenamiento topológico dentro del método HayCiclos(). En nuestro caso llevamos un contador de la cantidad de vértices procesados por el algoritmo, sin utilizar ninguna estructura auxiliar. Si al finalizar la ejecución del ordenamiento la cantidad de vértices procesados es igual a la cantidad de vértices totales del grafo, entonces quiere decir que no hay ciclos presentes.

Complejidad temporal: La complejidad es $O(V + E)$, recorriendo todos los vértices y sus aristas.

Estructuras utilizadas:

- Lista de adyacencias
- Ordenamiento topológico

Declaración de autoría: Declaramos que esta solución fue implementada por nuestro grupo de forma original.

Declaraciones generales y pruebas:

Para validar nuestras soluciones, desarrollamos un script automatizado que prueba cada ejercicio con todos los casos de entrada proporcionados por el docente.

El script compila el programa, lo ejecuta con cada archivo `.in.txt` y compara la salida generada con la salida esperada.

Este script fue creado con ayuda de ChatGPT, y nos permitió asegurar que los resultados obtenidos fueran correctos para todos los casos de prueba rápidamente.

Ejemplo del código del script realizado para el ejercicio 4:

```
tests > ejercicio4 > >_ automaticTests_ej4.sh
1
2  #CODIGO PARA TESTEAR AUTOMATICAMENTE TODOS LOS CASOS REALIZADO CON CHATGPT
3
4  #!/bin/bash
5  # Ir al directorio raíz del proyecto
6  cd ../../
7
8  # Compilar el ejercicio
9  g++ ejercicio4.cpp -o ejercicio4
10
11 # Probar todos los casos
12 for test in tests/ejercicio4/*.in.txt; do
13     base=$(basename $test .in.txt)
14     echo "Testing $base:"
15     ./ejercicio4 < $test > tests/ejercicio4/$base.mio.out.txt
16     ./compare.sh tests/ejercicio4/$base.mio.out.txt tests/ejercicio4/$base.out.txt
17     if [ $? -eq 0 ]; then
18         echo " Test $base passed"
19     else
20         echo " Test $base failed"
21     fi
22     echo "-----"
23 done
```

A continuación se presentan las pruebas realizadas para cada ejercicio utilizando el script automatizado previamente detallado.

Las capturas muestran la ejecución del script y los resultados obtenidos, validando que todas las soluciones pasaron correctamente los casos de prueba correspondientes a cada ejercicio.

Ejercicio 1 - Emails:

```
● → ejercicio1 git:(main) ✗ bash automaticTests_ej1.sh
Testing 10:
  Test 10 passed
-----
Testing 100:
  Test 100 passed
-----
Testing 1000:
  Test 1000 passed
-----
Testing 10000:
  Test 10000 passed
-----
Testing 100000:
  Test 100000 passed
-----
```

Ejercicio 2 - Diccionario:

```
● → ejercicio2 git:(main) ✗ bash automaticTests_ej2.sh
Testing dic10e1:
  Test dic10e1 passed
-----
Testing dic10e3:
  Test dic10e3 passed
-----
Testing dic10e6:
  Test dic10e6 passed
-----
```

Ejercicio 3 - Heapify:

```
● → ejercicio3 git:(main) ✗ bash automaticTests_ej3.sh
Testing 1000:
  Test 1000 passed
-----
Testing 10000:
  Test 10000 passed
-----
Testing 100000:
  Test 100000 passed
-----
Testing 1000000:
  Test 1000000 passed
-----
Testing 10000000:
  Test 10000000 passed
-----
Testing 120000:
  Test 120000 passed
-----
Testing 2000:
  Test 2000 passed
-----
Testing 20000:
  Test 20000 passed
-----
Testing 5000:
  Test 5000 passed
-----
Testing 50000:
  Test 50000 passed
-----
Testing 60000:
  Test 60000 passed
-----
Testing 70000:
  Test 70000 passed
-----
```


Ejercicio 4 - Camino más corto:

```
● → ejercicio4 git:(main) ✗ bash automaticTests_ej4.sh
Testing dijk10e1:
  Test dijk10e1 passed
=====
Testing dijk10e2:
  Test dijk10e2 passed
=====
```

Ejercicio 5 - Ciclicidad de grafos:

```
● → ejercicio5 git:(main) ✗ bash automaticTests_ej5.sh
Testing cycl10e1:
  Test cycl10e1 passed
=====
Testing cycl10e2:
  Test cycl10e2 passed
=====
```